

TRABAJO FINAL

Taller de Construcción de Software (INF243)

Universidad Siglo 21 — 2026

EcoMarket

Plataforma de Comercio Electrónico Sostenible

Integrantes

Cuberli, Roman — 45352533

Rinaldi, Nicolás — 45403417

Aguilar, Esteban — 46307516

Bruno, Enzo Eduardo — 38410678

Repositorio: github.com/nicote0/ecomarket

1. Introducción

EcoMarket es una aplicación web que simula el funcionamiento de una tienda online de productos ecológicos y sostenibles. El sistema permite consultar un catálogo de productos, administrar un carrito de compras y registrar órdenes una vez que el usuario confirma la compra. El proyecto se desarrolló con una arquitectura cliente-servidor, separando claramente el frontend del backend para facilitar el desarrollo y el mantenimiento.

El backend se implementó con Spring Boot, aprovechando su organización por capas y sus herramientas para construir APIs REST. El frontend se desarrolló con React, que consume la API mediante solicitudes HTTP e intercambia información en formato JSON. La persistencia se resolvió con Spring Data JPA sobre una base de datos MySQL.

A lo largo del proyecto se buscó aplicar los conceptos de la materia: separación de responsabilidades, reutilización de código y organización por capas. Este informe documenta la arquitectura, el modelo de datos, el diseño de la API, el desarrollo del frontend, la integración entre ambas partes y las decisiones técnicas adoptadas.

2. Arquitectura del sistema

EcoMarket sigue una arquitectura cliente-servidor con el backend organizado en capas. El frontend (React) envía solicitudes HTTP al backend; los controladores reciben las peticiones y las derivan a los servicios, que implementan la lógica de negocio; cuando se necesita acceder a los datos, los servicios utilizan los repositorios provistos por Spring Data JPA. La respuesta vuelve al frontend en formato JSON.

Flujo de una petición (de arriba hacia abajo):

- Frontend (React) — interfaz de usuario
- Controllers — reciben las peticiones HTTP
- Services — lógica de negocio
- Repositories — acceso a datos (Spring Data JPA)
- Base de datos (MySQL)

Esta separación permite mantener un código ordenado, reducir el acoplamiento y facilitar futuras modificaciones, ya que cada capa cumple una responsabilidad específica.

3. Modelo de datos

El modelo está compuesto por cuatro entidades JPA. Producto representa cada artículo del catálogo. Carrito agrupa los ítems que el usuario selecciona; cada ItemCarrito asocia un Producto con una cantidad. Al confirmar la compra, la Orden guarda una copia de los ítems junto con la fecha/hora, el total y el mensaje del cliente, de modo que el historial quede independiente de cambios posteriores en el carrito.

Entidad	Atributos principales	Relaciones
Producto	id, nombre, descripcion, precio, stock, categoria	Es referenciado por ItemCarrito

Carrito	id, items (lista de ItemCarrito)	1 a N con ItemCarrito (composición)
ItemCarrito	id, producto, cantidad	N a 1 con Producto; pertenece a un Carrito o a una Orden
Orden	id, fechaHora, mensajeCliente, total, items	Copia los ítems del carrito al confirmarse

Las relaciones se mapean con anotaciones JPA (@OneToMany y @ManyToOne). El total del carrito y de la orden se calcula como la suma de precio × cantidad de cada ítem.

4. Diseño de la API REST

La API expone los recursos productos, carritos y ordenes. Todos los endpoints responden en formato JSON y utilizan los códigos de estado HTTP correspondientes a cada operación (201 al crear, 204 al eliminar, 400 ante datos inválidos, 404 cuando el recurso no existe). Las entradas se validan con Bean Validation y los errores se devuelven de forma uniforme gracias a un manejador global de excepciones.

Método	Endpoint	Descripción	Códigos HTTP
GET	/api/productos	Listar el catálogo de productos	200
GET	/api/productos/{id}	Ver el detalle de un producto	200 / 404
POST	/api/productos	Crear un producto	201 / 400
PUT	/api/productos/{id}	Modificar un producto	200 / 400 / 404
DELETE	/api/productos/{id}	Eliminar un producto	204 / 404
POST	/api/carritos	Crear un carrito vacío	201
GET	/api/carritos/{id}	Ver el carrito con sus ítems	200 / 404
POST	/api/carritos/{id}/items	Agregar un ítem al carrito	200 / 400 / 404
PUT	/api/carritos/{id}/items/{itemId}	Modificar la cantidad de un ítem	200 / 400 / 404
DELETE	/api/carritos/{id}/items/{itemId}	Eliminar un ítem del carrito	200 / 404
POST	/api/ordenes/confirmar/{carritoid}	Confirmar el carrito como orden	201 / 400 / 404
GET	/api/ordenes	Listar el historial de órdenes	200
GET	/api/ordenes/{id}	Ver una orden	200 / 404

Antes de integrar el frontend, todos los endpoints se probaron con Postman. La colección exportada (EcoMarket.postman_collection.json) se incluye en el repositorio.

5. Desarrollo del backend

El backend se organiza en paquetes por capa. Los controladores (capa Controller) son el punto de entrada de las solicitudes; reciben la petición y la derivan al servicio correspondiente, sin incluir lógica de negocio. La capa Service concentra las reglas del sistema: administrar productos, gestionar el carrito y registrar las órdenes. La capa Repository, implementada con Spring Data JPA, resuelve el acceso a la base de datos sin escribir SQL para las operaciones habituales.

En el paquete model están las entidades Producto, Carrito, ItemCarrito y Orden. Para la comunicación con el cliente se utilizan DTOs (AgregarItemRequest, ModificarItemRequest y ConfirmarOrdenRequest), que transportan solo la información necesaria de cada operación y evitan exponer directamente las entidades. Finalmente, un GlobalExceptionHandler centraliza el manejo de excepciones y mantiene respuestas de error consistentes (por ejemplo, 400 ante una validación fallida o un carrito vacío).

6. Desarrollo del frontend

El frontend se construyó con React (y Vite como herramienta de desarrollo), utilizando exclusivamente componentes funcionales y los hooks useState y useEffect. useEffect se usa para la carga inicial de datos desde la API (productos, órdenes y carrito guardado) y useState para administrar el estado de la aplicación.

El flujo de datos respeta el patrón unidireccional de React: el estado principal (productos, carrito, órdenes y la vista actual) vive en el componente App. Ese estado se transmite a los componentes hijos mediante props, y las acciones del usuario se comunican hacia arriba mediante callbacks. Por ejemplo, ProductoCard recibe el producto por props y avisa "agregar al carrito" a través de una función recibida desde App.

Estructura principal de componentes:

- App — estado global y orquestación (useState / useEffect)
- Header — navegación entre catálogo, carrito y pedidos
- Catalogo / ProductoCard — listado y tarjeta de producto, con filtro por categoría
- ProductoDetalle — vista de detalle de un producto
- ProductoForm — alta y edición de productos (gestión del catálogo)
- Carrito — ítems, cantidades y total, con confirmación del pedido
- Pedidos — historial de órdenes confirmadas

El identificador del carrito se guarda en localStorage para que persista entre recargas; al confirmar un pedido se descarta y se crea uno nuevo. La capa api/api.js centraliza todas las llamadas a la API.

7. Integración y resolución de CORS

Problema:

El frontend se ejecuta en http://localhost:5173 (servidor de Vite) y el backend en http://localhost:8080. Al ser orígenes distintos, el navegador aplica la política Same-Origin y bloquea las solicitudes del frontend hacia la API (error de CORS), impidiendo que la aplicación obtenga los datos.

Solución:

Se habilitó CORS en el backend agregando la anotación `@CrossOrigin(origins = "**")` en los controladores de la API. De esta manera, las respuestas incluyen las cabeceras necesarias (`Access-Control-Allow-Origin`) y el navegador permite la comunicación entre el frontend y el backend. Con esa configuración, ambas partes funcionan de manera integrada durante la entrega.

8. Declaración de uso de inteligencia artificial

Conforme a lo solicitado en la consigna, se declara el uso de herramientas de inteligencia artificial generativa durante el desarrollo. La asistencia fue revisada, comprendida e integrada por el grupo, y cada integrante puede explicar el código correspondiente.

Parte del trabajo	Uso de la herramienta
Backend (pulido)	Validaciones (Bean Validation), códigos de estado HTTP, manejador global de excepciones, DTOs y datos de ejemplo. Asistencia revisada y comprendida por el grupo.
Colección Postman	Generación y ordenamiento de la colección de pruebas de los endpoints.
Frontend (React)	Asistencia en la estructura de componentes, el consumo de la API y el diseño de la interfaz.
Informe	Apoyo en la redacción y organización de algunas secciones de este documento.

Nota: ajustar esta tabla según las herramientas que cada integrante haya utilizado realmente (por ejemplo, ChatGPT, GitHub Copilot o Claude) y las partes en las que se aplicaron.

9. Conclusiones

El desarrollo de EcoMarket permitió construir una aplicación web funcional que integra un backend en Spring Boot y un frontend en React, cubriendo el flujo completo de una compra: consultar el catálogo, administrar el carrito y registrar órdenes.

Mantener una arquitectura por capas y aplicar buenas prácticas (DTOs, validación de entradas, manejo centralizado de excepciones y repositorios para el acceso a datos) dio como resultado un código ordenado, con bajo acoplamiento y fácil de mantener. La resolución de la integración mediante CORS permitió que ambas partes funcionen de manera conjunta.

Como resultado se obtuvo una aplicación estable que cumple los objetivos planteados. El proyecto deja una base sólida sobre la cual se podrían incorporar nuevas funcionalidades, y representó una oportunidad para poner en práctica criterios de organización y diseño de software aplicables a proyectos de mayor escala.

10. Reflexiones individuales

Cada integrante comparte una breve reflexión personal sobre el trabajo realizado.

Reflexión — Cuberli, Roman

[Completar — aproximadamente una página] Respondé con tus palabras:

- ¿Qué aprendiste durante el desarrollo del proyecto?
- ¿Qué harías diferente si lo volvieras a encarar?
- ¿Cuál fue tu principal aporte al grupo y en qué parte del código trabajaste?

Reflexión — Rinaldi, Nicolás

[Completar — aproximadamente una página] Respondé con tus palabras:

- ¿Qué aprendiste durante el desarrollo del proyecto?
- ¿Qué harías diferente si lo volvieras a encarar?
- ¿Cuál fue tu principal aporte al grupo y en qué parte del código trabajaste?

Reflexión — Aguilar, Esteban

[Completar — aproximadamente una página] Respondé con tus palabras:

- ¿Qué aprendiste durante el desarrollo del proyecto?
- ¿Qué harías diferente si lo volvieras a encarar?
- ¿Cuál fue tu principal aporte al grupo y en qué parte del código trabajaste?

Reflexión — Bruno, Enzo Eduardo

[Completar — aproximadamente una página] Respondé con tus palabras:

- ¿Qué aprendiste durante el desarrollo del proyecto?
- ¿Qué harías diferente si lo volvieras a encarar?
- ¿Cuál fue tu principal aporte al grupo y en qué parte del código trabajaste?